

GEMINI HISTORICAL ARTIFACT DESCRIPTOR

1. Introduction

The Gemini Historical Artifact Descriptor is an advanced multimodal artificial intelligence application designed to generate academic-style historical documentation from artifact images. The system leverages Google's Gemini 2.5 Flash model to analyze visual and textual inputs simultaneously. In traditional historical research workflows, artifact documentation involves manual observation, contextual research, material analysis, and structured writing. This process is time-intensive and requires subject-matter expertise. The proposed system automates this workflow by integrating computer vision capabilities with natural language generation.

This project demonstrates the interdisciplinary integration of artificial intelligence, digital humanities, web technologies, and secure API-based model deployment. It allows users to upload an artifact image, provide contextual instructions, and generate a structured 1000-word academic blog-style description. The generated content includes historical background, stylistic analysis, cultural context, comparative evaluation, and scholarly interpretation.

2. Problem Statement

Museums, academic institutions, and cultural heritage organizations manage large repositories of historical artifacts. Documenting each artifact manually requires historians to examine visual patterns, identify cultural markers, assess material composition, and research the socio-political context of the artifact's time period. Such manual documentation is not scalable for large archives.

Furthermore, students and researchers often struggle with structuring their artifact analysis into formal academic prose. The absence of automated documentation tools limits efficiency in digital archiving initiatives. Therefore, there is a need for a system that combines multimodal AI capabilities with an accessible web interface to generate structured historical narratives efficiently.

3. Objectives

The primary objective of this project is to design and develop a multimodal AI application capable of generating detailed academic descriptions of historical artifacts using image and text inputs.

Specific objectives include: 1) Integrating Google Gemini API for multimodal analysis. 2) Designing a user-friendly interface using Streamlit. 3) Ensuring secure API key management using environment variables. 4) Enforcing academic writing tone using system-level prompt engineering. 5) Generating structured outputs exceeding 1000 words with contextual depth.

4. Literature Review

Recent advancements in multimodal generative AI systems have enabled simultaneous understanding of textual and visual data. Transformer-based large language models have evolved from pure text-only systems to multimodal architectures capable of interpreting complex visual contexts.

The Gemini model family represents an evolution in this domain, offering high-speed inference and multimodal reasoning capabilities. Applications of such models have been observed in education, content generation, automated tutoring, digital design, and cultural heritage preservation.

5. Technologies Used

Python programming language was used due to its extensive AI ecosystem and simplicity. Streamlit serves as the web application framework that enables rapid UI development. The Google Generative AI SDK (google-genai) connects the application to the Gemini 2.5 Flash model. The python-dotenv library ensures API key confidentiality by loading sensitive credentials from a protected .env file. Pillow (PIL) is utilized to process and preview uploaded images.

The requirements.txt file manages project dependencies and ensures reproducible environments. This modular stack ensures scalability, maintainability, and secure integration.

6. System Architecture

The system follows a modular client-server architecture. The frontend layer, built using Streamlit, collects user input and artifact images. The backend logic initializes the Gemini client and sends multimodal inputs to the model using the generate_content method.

Data flow begins with user interaction. The uploaded artifact image and optional text query are bundled with a system instruction enforcing academic tone. The Gemini model processes this combined input and returns a structured textual response. The response is rendered dynamically in Markdown format on the web interface.

7. Implementation Details

The implementation begins by loading environment variables containing the API key. The genai.Client object is initialized with the secure key. A dedicated function handles model invocation by passing system prompts, user queries, and PIL image objects directly to the API.

Streamlit components such as text_input, file_uploader, button, spinner, and markdown are used to create interactive behavior. The spinner provides visual feedback during model processing. Conditional validation ensures that an image must be uploaded before processing begins.

8. Features of the System

Key features include multimodal reasoning, academic tone control through system prompts, secure credential management, dynamic web-based visualization, and structured long-form output generation.

The system supports contextual customization where users can ask the model to focus on specific historical eras, artistic movements, symbolism, or material analysis. This enhances usability in academic research contexts.

9. Advantages

The application significantly reduces time required for artifact documentation. It provides a scalable solution for digital museum archiving and educational assignments. The integration of AI enhances productivity in humanities research and digital preservation.

The modular architecture enables easy extension for cloud deployment and additional AI capabilities.

10. Limitations

The system relies on AI-generated interpretation, which may sometimes include factual inaccuracies. It requires internet connectivity and API access. The model does not replace domain experts but assists in drafting.

11. Future Enhancements

Future scope includes citation auto-generation, integration with artifact databases, multilingual support, cloud deployment, user authentication, logging mechanisms, and offline archival export functionality.

12. Conclusion

The Gemini Historical Artifact Descriptor demonstrates the powerful intersection of multimodal AI and digital humanities. By automating artifact documentation through structured academic writing, the project highlights practical applications of generative AI in research, education, and cultural preservation.

The successful integration of Streamlit and Google Gemini API establishes a scalable framework for future AI-driven historical analysis systems.

13. APPENDEX

13.1 Source Code

.env:

```
GOOGLE_API_KEY="AIzaSyAGcv3ikeCv1Wtt_LP90yoql-nKDLyCIiw"
```

Requirements.txt:

```
streamlit  
google-genai  
python-dotenv  
Pillow
```

App.py:

```
import streamlit as st  
import os  
from google import genai  
from PIL import Image  
from dotenv import load_dotenv  
  
# Load Environment & Setup Client  
load_dotenv()  
# The new SDK uses a Client object for all interactions  
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))  
  
# Use the latest stable "Flash" model as of Feb 2026  
MODEL_ID = "gemini-2.5-flash"  
  
def get_gemini_response(user_input, image, system_prompt):  
    """Uses the new SDK to generate multimodal content."""  
    # The new SDK structure: client.models.generate_content  
    response = client.models.generate_content(  
        model=MODEL_ID,  
        contents=[system_prompt, user_input, image]  
    )  
    return response.text  
  
# --- Streamlit UI ---  
st.set_page_config(page_title="Historical Explorer", page_icon="📜")  
st.header("Gemini historical artifact description")
```

```
user_prompt = st.text_input("Enter Prompt:")
uploaded_file = st.file_uploader("Upload Artifact Image", type=["jpg", "png",
"jpeg"])

if uploaded_file:
    # Open image for UI display
    img = Image.open(uploaded_file)
    st.image(img, caption="Target Artifact", use_container_width=True)

if st.button("Generate Description"):
    if uploaded_file:
        with st.spinner("Gemini is analyzing the artifact..."):
            # Pass the PIL Image directly; the new SDK handles it!
            system_instruction = "You are a historian. Please describe the
historical artifact in the image and provide detailed information, including
its name, origin, time period , historical significance,interesting facts"
            description = get_gemini_response(user_prompt, img,
system_instruction)

            st.subheader("Historical Description")
            st.markdown(description)
    else:
        st.error("Please upload an image first.")
```